
Data Structures

BS (CS) _Fall_2025

Lab_10 Task



Learning Objectives:

1. Binary Trees
2. Array to Binary tree and Binary Tree to Array

Task 1: Organizational Hierarchy Tree (Level-Order Binary Tree)

Objective

- To implement a Level-Order Binary Tree that models a company's reporting hierarchy.
- You will practice pointer manipulation, iterative insertion/deletion, and relationship queries in a real-world structure.

Scenario

You are working with TechNova Pvt. Ltd., where the HR department wants to visualize and manage the organizational hierarchy using a binary tree.

Each node represents an Employee, and the tree must always remain complete — filled from left to right, top to bottom.

Each employee record will include:

- Employee ID (int, unique)
- Name (string)
- Designation (string) (e.g., CEO, Manager, Engineer, Intern)
- Department (string)

The root node represents the CEO, and every employee can have up to two subordinates (left and right).

Functional Requirement

You are required to design a class-based system to perform the following operations:

- 1. Add Employee (Level-Order Insertion)**
 - Insert new employees in level order (left to right).
 - The first record added becomes the CEO (root node).
 - Maintain the complete binary tree property.
- 2. Display Hierarchy**
 - Level-Order: Top to bottom (breadth-wise view)
- 3. Each display should print:**
 - [ID | Name | Designation | Department]
- 4. Update Employee Information**
 - Search an employee by ID.
 - Allow updating one or more fields (Name, Designation, or Department).
- 5. Delete Employee**
 - Delete an employee by Employee ID.
 - Replace the deleted node with the deepest and rightmost node to keep the tree complete.
- 6. Search Employee**

- Search an employee by ID or Name.
- Display full details if found; otherwise show “Employee not found.”

```
#include <gtest/gtest.h>
#include <iostream>
#include <string>
using namespace std;

// Include the student's code file
#include "OrganizationTree.cpp"

// ----- TEST CASES -----

TEST(OrganizationTreeTest, AddEmployee_LevelOrderInsertion) {
    OrganizationTree org;

    org.addEmployee(1, "Alice", "CEO", "Management");
    org.addEmployee(2, "Bob", "Manager", "Engineering");
    org.addEmployee(3, "Carol", "Manager", "HR");
    org.addEmployee(4, "Dave", "Engineer", "Backend");

    // Check root and structure
    EmployeeNode* root = org.searchByID(1);
    ASSERT_NE(root, nullptr);
    EXPECT_EQ(root->name, "Alice");

    // Left and right of root should be filled first
    EXPECT_NE(root->left, nullptr);
    EXPECT_NE(root->right, nullptr);

    EXPECT_EQ(root->left->name, "Bob");
    EXPECT_EQ(root->right->name, "Carol");

    // Next insertion should go under Bob (left)
    EXPECT_EQ(root->left->left->name, "Dave");
}

TEST(OrganizationTreeTest, DisplayHierarchy_LevelOrder) {
    OrganizationTree org;

    org.addEmployee(1, "Alice", "CEO", "Management");
    org.addEmployee(2, "Bob", "Manager", "Engineering");
    org.addEmployee(3, "Carol", "Manager", "HR");
```

```

    // Display should print all employees
    cout << "\nExpected output format: [ID | Name | Designation | Department]\n";
    org.displayHierarchy();

    // Just check if nodes exist in tree
    EXPECT_NE(org.searchByName("Alice"), nullptr);
    EXPECT_NE(org.searchByName("Bob"), nullptr);
    EXPECT_NE(org.searchByName("Carol"), nullptr);
}

TEST(OrganizationTreeTest, SearchEmployee_ByIDandName) {
    OrganizationTree org;

    org.addEmployee(1, "Alice", "CEO", "Management");
    org.addEmployee(2, "Bob", "Manager", "Engineering");
    org.addEmployee(3, "Carol", "Manager", "HR");

    EmployeeNode* e1 = org.searchByID(2);
    EXPECT_NE(e1, nullptr);
    EXPECT_EQ(e1->name, "Bob");

    EmployeeNode* e2 = org.searchByName("Carol");
    EXPECT_NE(e2, nullptr);
    EXPECT_EQ(e2->designation, "Manager");

    // Non-existing cases
    EXPECT_EQ(org.searchByID(100), nullptr);
    EXPECT_EQ(org.searchByName("Zara"), nullptr);
}

TEST(OrganizationTreeTest, UpdateEmployee_ChangesData) {
    OrganizationTree org;

    org.addEmployee(1, "Alice", "CEO", "Management");
    org.addEmployee(2, "Bob", "Manager", "Engineering");

    EmployeeNode* emp = org.searchByID(2);
    ASSERT_NE(emp, nullptr);

    // Manually update (simulate user input)
    emp->name = "Bobby";
    emp->designation = "Senior Manager";
    emp->department = "Tech";

    EXPECT_EQ(emp->name, "Bobby");
}

```

```

    EXPECT_EQ(emp->designation, "Senior Manager");
    EXPECT_EQ(emp->department, "Tech");
}

TEST(OrganizationTreeTest, DeleteEmployee_ReplacesWithDeepest) {
    OrganizationTree org;

    org.addEmployee(1, "Alice", "CEO", "Management");
    org.addEmployee(2, "Bob", "Manager", "Engineering");
    org.addEmployee(3, "Carol", "Manager", "HR");
    org.addEmployee(4, "Dave", "Engineer", "Backend");

    // Delete Bob
    org.deleteEmployee(2);

    // Bob should be replaced by the deepest (Dave)
    EmployeeNode* e = org.searchByName("Dave");
    EXPECT_NE(e, nullptr); // Dave still exists (moved)
    EXPECT_EQ(org.searchByName("Bob"), nullptr); // Bob deleted
}

TEST(OrganizationTreeTest, DeleteRoot_RemovesWholeTree) {
    OrganizationTree org;
    org.addEmployee(1, "Alice", "CEO", "Management");

    org.deleteEmployee(1);
    EXPECT_EQ(org.searchByID(1), nullptr);
}

```

Task 2: Constructing and Converting Binary Trees (Array ↔ Tree)

Objective:

Understand how binary trees can be represented using arrays and vice versa.

Practice index relationships between parent and child nodes.

Implement level-order construction and traversal using pointers.

Scenario

The university's database team stores student roll numbers in a flat array format, but the research department wants to visualize them as a binary tree for structural analysis.

You are tasked to:

- Construct a Binary Tree from a given array (using level-order logic).
- Convert the constructed Binary Tree back into an array form.
- Display both representations clearly.

Functional Requirements

Implement a class BinaryTree with the following functionalities:

- 1. Construct Tree from Array**
 - Take an integer array as input.
 - Build the tree in level-order using the index formula.
- 2. Display Tree (Level-Order)**
 - Display all nodes breadth-wise.
 - Example output:
10 20 30 40 50 60
- 3. Convert Tree to Array**
 - Perform a level-order traversal of the tree.
 - Store all nodes back into an array.
 - Print the resulting array.
- 4. Count and Display Nodes**
 - Count total nodes and display the number.

```
#include <gtest/gtest.h>
#include <iostream>
using namespace std;

// Include the student's main code file
#include "BinaryTree.cpp"

// ----- TEST CASES -----

// Construct tree from array (normal case)
TEST(BinaryTreeTest, ConstructTreeFromArray_Normal) {
    BinaryTree tree;
    int arr[] = {10, 20, 30, 40, 50, 60};
    int n = sizeof(arr)/sizeof(arr[0]);

    Node* root = tree.buildFromArray(arr, n);
    ASSERT_NE(root, nullptr);

    // Validate structure
    EXPECT_EQ(root->data, 10);
```

```

    EXPECT_EQ(root->left->data, 20);
    EXPECT_EQ(root->right->data, 30);
    EXPECT_EQ(root->left->left->data, 40);
    EXPECT_EQ(root->left->right->data, 50);
    EXPECT_EQ(root->right->left->data, 60);
}

// Display tree (level-order)
TEST(BinaryTreeTest, DisplayTreeLevelOrder) {
    BinaryTree tree;
    int arr[] = {1, 2, 3, 4, 5};
    int n = sizeof(arr)/sizeof(arr[0]);
    tree.buildFromArray(arr, n);

    cout << "\nExpected Level Order: 1 2 3 4 5\n";
    tree.displayLevelOrder(); // Visible for students
    SUCCEED(); // passes if no crash
}

// Convert tree back to array (check traversal order)
TEST(BinaryTreeTest, ConvertTreeToArray) {
    BinaryTree tree;
    int arr[] = {11, 22, 33, 44, 55};
    int n = sizeof(arr)/sizeof(arr[0]);
    tree.buildFromArray(arr, n);

    cout << "\nExpected Array: [11 22 33 44 55]\n";
    tree.toArray();
    SUCCEED();
}

// Count and display total nodes
TEST(BinaryTreeTest, CountNodes) {
    BinaryTree tree;
    int arr[] = {5, 10, 15, 20};
    int n = sizeof(arr)/sizeof(arr[0]);
    tree.buildFromArray(arr, n);

    EXPECT_EQ(tree.countNodes(tree.getRoot()), 4);
}

// Empty array input should handle safely
TEST(BinaryTreeTest, HandlesEmptyArray) {
    BinaryTree tree;
    int arr[] = {};

```

```

    int n = 0;

    Node* root = tree.buildFromArray(arr, n);
    EXPECT_EQ(root, nullptr);

    EXPECT_EQ(tree.countNodes(tree.getRoot()), 0);

    cout << "\nShould print 'Tree is empty.' twice below:\n";
    tree.displayLevelOrder();
    tree.toArray();
    SUCCEED();
}

// Single element array (tree with only root)
TEST(BinaryTreeTest, SingleElementArray) {
    BinaryTree tree;
    int arr[] = {42};
    int n = 1;
    Node* root = tree.buildFromArray(arr, n);

    ASSERT_NE(root, nullptr);
    EXPECT_EQ(root->data, 42);
    EXPECT_EQ(root->left, nullptr);
    EXPECT_EQ(root->right, nullptr);
    EXPECT_EQ(tree.countNodes(root), 1);
}

// Uneven array (incomplete last level)
TEST(BinaryTreeTest, UnevenArrayIncompleteLastLevel) {
    BinaryTree tree;
    int arr[] = {1, 2, 3, 4, 5, 6};
    int n = 6;
    Node* root = tree.buildFromArray(arr, n);

    // last rightmost child of right->right should be nullptr
    EXPECT_EQ(root->right->right, nullptr);
    EXPECT_EQ(tree.countNodes(root), 6);
}

// Duplicate values (tree should still build correctly)
TEST(BinaryTreeTest, HandlesDuplicateValues) {
    BinaryTree tree;
    int arr[] = {10, 10, 10, 20, 20};
    int n = sizeof(arr)/sizeof(arr[0]);
    Node* root = tree.buildFromArray(arr, n);

```



```

    ASSERT_NE(root, nullptr);
    EXPECT_EQ(root->data, 10);
    EXPECT_EQ(root->left->data, 10);
    EXPECT_EQ(root->right->data, 10);
    EXPECT_EQ(tree.countNodes(root), 5);
}

// Large array test (stress check)
TEST(BinaryTreeTest, LargeArrayStressTest) {
    BinaryTree tree;
    const int n = 50;
    int arr[n];
    for (int i = 0; i < n; i++) arr[i] = i + 1;

    Node* root = tree.buildFromArray(arr, n);
    EXPECT_NE(root, nullptr);
    EXPECT_EQ(tree.countNodes(root), n);
}

```